

Fig. 3: An actual-size, single-side PCB for the serial communications with AVR microcontroller

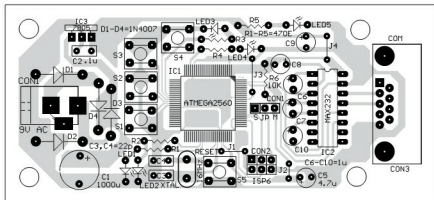


Fig. 4: Component layout for the PCB

clock signal generated by the master. Fig. 1 shows SPI master-slave interconnection.

Once the 8-bit data transfer is complete, SPI interrupt flag (SPIF) is set high in SPI status register (SPSR) and the data is available to be read in SPI data register (SPDR) of the slave. Similarly, when the data is copied to SPDR in master, it automatically starts transferring cyclically through its shift register to the slave. As soon as the 8-bit data is transferred, SPIF flag in SPSR is set.

To initialise a device as master, the MOSI, SCK and SS pins are set as output by setting the corresponding bits high in data direction register (DDRX). In SPI control register (SPCR), SPI enable (SPE), SPI clock rate (SPR0-SPR1) bits are set high. Clock rate by the master should be such that low and high periods are longer than two CPU clock cycles for the slave. Master ad-

ditionally drives the slave's $\overline{SS}$ pin low. To initialise a device as slave, the MISO pin in DDRx and SPE pin in SPCR are set high.

Circuit description

Fig. 2 shows the synchronous serial communication with AVR microcontroller. At the heart of the circuit is ATmega2560 (IC1), which is a low-power CMOS 8-bit microcontroller, based on the AVR enhanced RISC architecture. It features 128kB Flash memory with read-while-write capabilities, 4kB EEPROM, 8kB SRAM, 86 general-purpose input/output lines, 32 general-purpose working registers, six flexible timers/counters with compare modes, real time counter (RTC), 10-bit analogue-to-digital converter, internal and external interrupts, a serial programmable USART, a programmable watchdog timer with internal oscillator and an SPI serial port.

Let's assume that IC1 (ATme-

ga2560) in Fig. 2 is a master device. We need another circuit same as Fig. 2 for slave device as well. Communications between master and slave devices are done through ISP6 connector as explained below.

Switches S1 through S4 are interfaced with port PJ0 though PJ3 of microcontroller IC1 respectively. Switches S1 through S4 are used as input control switches to turn off the LEDs (LED1 through LED4) connected in the slave microcontroller. LED1 through LED4 act as visual display for the device. LED1 through LED4 are interfaced with port pins PL6, PL7, PJ4 and PJ7, respectively. Switch S5 is used for manual reset. Port pins PB2 (MOSI), PB3 (MISO) and PB1 (SCK) are used for in-system programming (ISP). IC MAX232 (IC2) is used for serial communication. T_{XD} and R_{XD} pins of the microcontroller too are used for serial communication with the help of COM port. System clock plays a significant role in the operation of the microcontroller. A 16MHz quartz crystal provides basic clock to the microcontroller (IC1) at its pins 33 and 34.

The 230V, 50Hz AC mains is stepped down by transformer X1 to deliver a secondary output of 9V, 500mA. The transformer output is rectified by a full-wave rectifier comprising diodes D1 through D4, filtered by capacitor C1 and regulated by IC 7805 (IC3). Capacitor C2 bypasses the ripples present in regulated supply. LED5 acts as the power indicator and resistor R5 limits the current through LED5.

An actual-size, single-side PCB for the circuit of synchronous serial communication with AVR microcontroller is shown in Fig. 3 and its component layout in Fig. 4. Assemble the circuit on a PCB to minimise time and assembly errors. Carefully assemble the components and double-check for any overlooked error. Assemble two boards—one for master and another for slave. Burn the hex codes of the C source codes listed at the end of this